

IF2224: Teori Bahasa Formal dan Automata
Pembuatan Intermediate Code & Interpreter
Untuk Bahasa Pemrograman Arion



Dipersiapkan oleh:

13524008	Agatha Tatianingseto
13524030	Irvin Tandiarrang Sumual
13524074	Bernhard Aprillio Pramana
13524096	Moreno Syawali Ganda Sugita
13524110	Jennifer Khang

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025

Bab 1

Deskripsi Tugas

Pada tugas besar Milestone 4 ini, fokus utama pengembangan berada pada tahap akhir kompilasi bahasa pemrograman Arion, yaitu proses translasi dan eksekusi program. Sistem yang dibangun diharapkan untuk mampu memproses masukan berupa *Decorated Abstract Syntax Tree* (AST) yang telah divalidasi kebenaran tipe datanya melalui fase analisis semantik pada tahapan sebelumnya. Untuk mengimplementasi hal tersebut, pengembangan dibagi menjadi dua komponen arsitektural utama, yakni *Intermediate Code Generator* yang bertugas sebagai penerjemah struktur pohon menjadi instruksi linear, serta sebuah *Stack Machine* yang berperan sebagai *Interpreter* untuk mengeksekusi instruksi-instruksi tersebut secara langsung.

Komponen *Intermediate Code Generator* dibangun menggunakan pendekatan algoritma berbasis *Depth First Search* (DFS) yang digunakan untuk menelusuri setiap simpul pada *Decorated AST*. Selama proses penelusuran ini, generator akan memetakan setiap deklarasi variabel, operasi matematika, pemanggilan fungsi, hingga logika *control flow* menjadi serangkaian kode perantara. Kode yang dihasilkan berupa kumpulan perintah terstandarisasi, seperti perintah alokasi memori, pemuatan nilai (*load*), penyimpanan nilai (*store*), hingga instruksi lompatan bersyarat. Kumpulan instruksi ini dirancang secara independen agar tidak lagi terikat pada struktur sintaksis bahasa Arion, melainkan berupa langkah-langkah operasional yang siap dibaca oleh mesin eksekusi.

Setelah representasi instruksi berhasil dibentuk, komponen *Interpreter* akan mengambil alih siklus operasional program dengan memanfaatkan struktur memori utama berupa *stack*. Mesin eksekusi ini bekerja dengan cara membaca setiap instruksi secara sekuensial dan mengelola perubahan memori secara dinamis saat *runtime*. Pengelolaan memori dalam mesin berbasis *stack* ini sangat krusial, karena ia tidak hanya bertanggung jawab atas penyimpanan nilai literal dan variabel, tetapi juga harus mengatur pembangunan dan pembongkaran blok memori khusus (*stack frame*) setiap kali terjadi transisi atau pemanggilan antar subprogram.

Tujuan akhir dari perancangan kedua subsistem ini adalah untuk merealisasikan kemampuan kompilator agar dapat memproses kode sumber hingga menghasilkan keluaran final yang nyata. Saat *Interpreter* memproses instruksi yang berkaitan dengan fungsi bawaan

keluaran program, seperti operasi pencetakan, hasil evaluasi dari dalam tumpukan memori akan diekstrak dan ditampilkan ke layar pengguna.

Bab 2

Teori Singkat

2.1. *Arsitektur Compiler Back-end*

Dalam perancangan *compiler* dan *interpreter* modern, sistem secara garis besar dibagi menjadi dua fase utama, yaitu fase *Front-end* dan *Back-end*. Fase *Front-end* mencakup analisis leksikal, sintaksis, dan semantik yang berurusan langsung dengan struktur teks bahasa sumber untuk menghasilkan *Decorated Abstract Syntax Tree* (AST). Pada tahapan eksekusi dan translasi ini, fokus pengembangan sepenuhnya berada di fase *Back-end*. Fase *Back-end* bekerja secara independen dari sintaksis tata bahasa kode program, dan murni berfokus pada dua tahapan utama:

- Sintesis

Mengonversi struktur *Decorated* AST menjadi set instruksi operasional standar yang linie (*Intermediate Code*).

- Eksekusi

Mengelola memori serta menjalankan set instruksi tersebut secara berurutan menggunakan sebuah mesin virtual (*Interpreter*).

Pemisahan arsitektur ini (*separation of concerns*) dirancang agar mesin penerjemah menjadi modular. Jika di masa depan terjadi perubahan sintaksis pada bahasa sumber, modul mesin virtual (*Back-end*) tidak perlu mengalami perubahan selama *Front-end* tetap menghasilkan representasi instruksi yang kompatibel.

2.2. Konsep *Intermediate Code* (IC)

Intermediate Code adalah representasi instruksi program tingkat rendah yang bertindak sebagai jembatan antara *source code* dan bahasa mesin aktual. Pada pengembangan *compiler* Arion, *Intermediate Code* dirancang khusus menggunakan struktur perintah *Three-Address Code* (TAC) yang telah diadaptasi untuk memenuhi konvensi *Stack Machine*.

Proses pembentukan *Intermediate Code* atau *Intermediate Code Generation* dilakukan menggunakan pendekatan algoritma *Depth First Search* (DFS). Komponen generator akan menelusuri setiap simpul pada AST hingga ke simpul daun,

mengevaluasinya, lalu menghasilkan urutan instruksi ke dalam sebuah *array* memori. Setiap instruksi dalam kompilator ini direpresentasikan dengan format baku yang terdiri dari tiga atribut utama, yaitu

- **OpCode (Kode Operasi)**
Mendefinisikan jenis perintah yang harus dieksekusi (contoh: LIT untuk memuat nilai literal, INT untuk menginisiasi ruang memori, dan OPR untuk melakukan operasi perhitungan).
- **Level (Kedalaman Lingkup)**
Penanda yang melacak kedalaman tingkat pemanggilan statik (digunakan terutama saat berurusan dengan subprogram atau fungsi bersarang).
- **Operand**
Menyimpan informasi nilai konkret, parameter nomor operasi pendukung (seperti fungsi turunan dari OPR), atau alamat memori *stack* yang dituju.

2.3. Arsitektur Eksekusi Berbasis Stack Machine

Interpreter yang dirancang untuk menjalankan instruksi *Intermediate Code* Arion dibangun di atas arsitektur *Stack Machine*. *Stack Machine* adalah jenis arsitektur komputer virtual di mana memori utamanya dimanipulasi menggunakan struktur data *stack* dengan prinsip LIFO (*Last In, First Out*). Dalam arsitektur ini, mesin tidak bergantung pada register fisik perangkat keras secara langsung, melainkan menggunakan memori dinamis untuk menyimpan variabel lokal dan melakukan evaluasi ekspresi matematis.

Mekanisme kerja *Interpreter* berbasis *stack* ini melibatkan *loop* dari tahapan *Fetch-Decode-Execute*:

1. Instruksi Pointer

Mesin menyimpan sebuah variabel pelacak yang berfungsi sebagai penunjuk baris (*Instruction Pointer*). IP bertugas memastikan instruksi mana di dalam larik kode perantara yang sedang atau akan dieksekusi selanjutnya.

2. Modifikasi Stack

Ketika mesin mengeksekusi operasi matematika atau relasional (misalnya ADD atau MUL pada instruksi OPR), mesin akan melakukan *pop* dua nilai teratas dari *stack* memori, memproses hasil operasi dari kedua nilai tersebut secara logis, dan kemudian melakukan *push* hasilnya ke *stack*.

3. Manajemen Memori

Alokasi memori untuk variabel diatur berdasarkan indeks alamat di dalam memori statis *stack*. Setiap fungsi pembacaan data (LOD) atau penyimpanan data (STO) akan dieksekusi dengan merujuk pada alamat relatif (*offset*) dari penanda memori (*base pointer*) saat itu.

2.4. Polimorfisme Memori dengan Type-Safe Union

Pada arsitektur *stack machine* klasik seperti P-Code Machine, *runtime stack* umumnya dirancang secara homogen untuk menampung satu tipe data primitif (misalnya representasi *integer*). Untuk mendukung eksekusi tipe data dinamis (*integer*, *real*, *char*, dan *string*) secara bersamaan pada *stack* yang sama tanpa alokasi *heap* manual, pendekatan *Type-Safe Union* digunakan. Implementasi di C++ modern difasilitasi oleh `std::variant`, yang memungkinkan sebuah sel memori menyimpan salah satu dari beberapa tipe data yang telah didefinisikan secara statis. Berbeda dengan `void*` atau *union* pada bahasa C konvensional, penanganan *type-safe union* memastikan program tidak mengalami *undefined behavior* saat membaca memori, karena tipe data aktifnya dilacak secara internal dan divalidasi saat *runtime*.

2.5. Resolusi Tipe Data Dinamis pada *Arithmetic Logic Unit* (ALU)

Dengan adanya memori yang heterogen, ALU pada mesin virtual perlu mengadopsi resolusi tipe dinamis. ALU tidak lagi secara naif memproses instruksi aritmatika (seperti instruksi ADD), melainkan melakukan ekstraksi (pop) dari *stack*, mengevaluasi tipe data operan, dan mengeksekusi operasi yang bersesuaian:

1. String Concatenation & Relational

Jika operan berupa *string* atau kombinasi *string* dan *char*, instruksi aritmatika seperti penjumlahan (+), serta operator relasional (=, <, >, dll.) dieksekusi sebagai *concatenation* dan perbandingan leksikografis standar.

2. Mixed Arithmetic

Jika ALU mendeteksi pertemuan antara *integer* dan *real*, promosi tipe data secara otomatis dilakukan ke *real* sebelum operasi hitung dijalankan untuk mencegah *precision loss*. Pembagian difurkasi menjadi dua instruksi independen: DIV untuk pembagian basis bulat dan RDIV untuk pembagian riil murni.

2.6. Keamanan Memori pada *Runtime Stack*

Dalam arsitektur perangkat lunak yang aman, mesin virtual harus kebal terhadap anomali instruksi yang merusak ruang alamat memori program. Keamanan ini direalisasikan dengan menerapkan mekanisme *bounds checking* yang ketat pada operasi *push*, *pop*, *load*, dan *store*:

- Stack Overflow

Deteksi proaktif saat alokasi memori melampaui ukuran maksimum memori yang diizinkan (`MAX_STACK_SIZE`).

- Stack Underflow

Proteksi agar instruksi operasi tidak mengambil data (*pop*) pada *stack* yang kosong.

- Out-of-Bounds Memory Access

Pemeriksaan alamat relatif dan absolut selama instruksi pemanggilan alamat leksikal (*Static/Dynamic Link*) agar *store/load* tidak menimpa memori terlarang di luar ruang *Stack Frame* yang sah. Seluruh pelanggaran ini ditangani menggunakan *Exception Handling* alih-alih memberhentikan proses dengan *Segmentation Fault*.

Bab 3

Perancangan dan Implementasi

3.1. *Intermediate Code Generation*

Fase ini bertugas menerjemahkan *Abstract Syntax Tree* (AST) dari fase semantik menjadi urutan instruksi linier berbasis *Stack Machine* atau P-Code.

3.1.1. Arsitektur Instruksi (P-Code)

Instruction.hpp

```
class Instruction {
private:
    OpCode op_;
    int level_;
    std::variant<int, double, char, std::string>
operand_;

public:
    Instruction(OpCode op, int level, std::variant<int,
double, char, std::string> operand);
    // ...
};
```

Instruksi direpresentasikan melalui kelas objek independen yang menyimpan OpCode (jenis instruksi seperti LIT, LOD, STO, CAL), Level (perbedaan kedalaman leksikal), dan Operand. Untuk mendukung tipe data heterogen tanpa kehilangan presisi, operand diimplementasikan menggunakan tipe dinamis yang dapat menyimpan bilangan bulat, desimal, karakter, maupun teks secara bersamaan.

Untuk mendukung akses elemen *array* secara dinamis saat eksekusi (*runtime*), himpunan instruksi P-Code diperluas dengan menambahkan instruksi LDA (*Load Address*) untuk memuat alamat memori ke *stack*, LDI (*Load Indirect*) untuk membaca nilai berdasarkan alamat di *stack*, STI (*Store Indirect*) untuk menyimpan nilai ke alamat referensi, serta CHK (*Check Bounds*) untuk memvalidasi batas indeks.

3.1.2 Manajemen Pengalamatan Memori (*Addressing*)

IntermediateCode.cpp


```

auto it = relativeOffsetMap_.find(node->symbolRefIndex_);
if (it != relativeOffsetMap_.end()) {
    runtimeAddress = it->second; // Bisa negatif
(parameter) atau positif (lokal)
}

if (it != relativeOffsetMap_.end() && it->second < 0) {
    // Parameter yang berhasil dipetakan secara negatif
    runtimeAddress = it->second;
} else if (it != relativeOffsetMap_.end() && it->second >=
FRAME_HEADER_SIZE) {
    // Variabel lokal
    runtimeAddress = it->second;
} else {
    // Fallback untuk Global Variable (Jika belum masuk
map)
    const IdentifierTableEntry& entry =
symbolTable_.getIdentifier(node->symbolRefIndex_);

    if (entry.level > 0 && entry.address < 10) {
        // Parameter paksa menjadi negatif
        runtimeAddress = -(entry.address + 1);
    } else {
        // Variabel global / lokal
        runtimeAddress = FRAME_HEADER_SIZE + entry.address;
    }
}
}

```

Menentukan alamat memori yang presisi untuk setiap variabel (*local*, *global*, maupun *parameter*) merupakan tantangan terbesar dan paling kompleks dalam fase ini. Implementasinya menggunakan kombinasi *Symbol Table* dan pemetaan *offset* relatif. Variabel lokal dihitung posisinya setelah memberikan ruang untuk *Frame Header* (ukuran 3 slot: *Static Link*, *Dynamic Link*, *Return Address*), sedangkan parameter disesuaikan posisinya dengan nilai *offset* negatif.

Mengakses elemen *array* menggunakan variabel indeks (misal: `arr[i]`) tidak dapat diselesaikan secara statis saat kompilasi (*compile-time*). Oleh karena itu, kompilator mengimplementasikan *dynamic addressing*. Generator kode akan memancarkan instruksi LDA untuk menempatkan alamat dasar (*base address*) *array* di *stack*. Nilai indeks kemudian dievaluasi, disesuaikan dengan batas bawah (*low bound*), dikalikan dengan ukuran elemen, dan ditambahkan ke alamat dasar. Eksekusi akhir pembacaan atau penulisan data dilakukan menggunakan instruksi LDI atau STI.

3.1.3 Translasi Node dan *Backpatching*

IntermediateCode.cpp

```
void IntermediateCodeGenerator::patchPendingCalls() {
    for (const PendingCallPatch& patch :
pendingCallPatches_) {
        auto it =
subprogramEntryLineBySymbolIndex_.find(patch.symbolIndex);
        if (it == subprogramEntryLineBySymbolIndex_.end())
        {
            throw std::runtime_error(
                "Intermediate Error: alamat subprogram '" +
patch.calleeName + "' belum diketahui."
            );
        }
        patchOperand(patch.instructionIndex, it->second);
    }
    pendingCallPatches_.clear();
}
```

Setiap *node* AST dikunjungi menggunakan pola *Visitor Pattern*. Untuk menangani pemanggilan fungsi/prosedur yang dideklarasikan di bagian bawah (*forward reference*), kompilator mengimplementasikan mekanisme *Pending Call Patching*. Alamat target lompatan (JMP/CAL) akan dicatat dan di-*patch* pada akhir proses pembangkitan kode setelah seluruh posisi baris subprogram diketahui.

3.2. Arsitektur Mesin Virtual (*Interpreter*)

Mesin virtual dirancang untuk mengeksekusi instruksi P-Code secara berurutan dalam sebuah lingkungan yang terisolasi dan aman.

3.2.1 Lingkungan Eksekusi (*Runtime Stack*)

RuntimeStack.hpp

```
class RuntimeStack {
private:
    std::vector<std::variant<int, double, char,
std::string>> stack_;
    int basePtr_;
    const size_t MAX_STACK_SIZE = 10000; // Batas aman
maksimum ukuran Stack
}
```

Memori utama direpresentasikan menggunakan *Runtime Stack* dinamis yang menyimpan data gabungan (*polymorphic*). Setiap kali prosedur baru dipanggil,

sebuah *Activation Record* (Stack Frame) baru dibuat dengan memindahkan *Base Pointer*. Penelusuran variabel di luar cakupan lokal dilakukan dengan melompati *Static Link* sesuai dengan selisih level leksikalnya.

3.2.2 Siklus Eksekusi (*Fetch-Decode-Execute*)

Interpreter menggunakan *loop* utama yang dikendalikan oleh *Instruction Pointer* (IP). Setiap iterasi akan melakukan proses *Fetch* (mengambil instruksi pada IP saat ini), *Decode* (membaca OpCode), dan *Execute* (menjalankan mutasi *stack* atau manipulasi IP seperti pada operasi JMP dan JPC).

3.2.3 *Arithmetic Logic Unit* (ALU) Fleksibel

ALU.cpp

```
case OprCode::ADD: {
    if (hasString) {
        std::string lStr, rStr;
        if (isString(valLeft)) lStr =
std::get<std::string>(valLeft);
        else if (std::holds_alternative<char>(valLeft))
lStr = std::string(1, std::get<char>(valLeft));

        if (isString(valRight)) rStr =
std::get<std::string>(valRight);
        else if (std::holds_alternative<char>(valRight))
rStr = std::string(1, std::get<char>(valRight));

        memory.push(lStr + rStr);
    } else if (hasDouble) {
        memory.push(getAsDouble(valLeft) +
getAsDouble(valRight)); // Hasil Double
    } else {
        long long res = static_cast<long
long>(getAsDouble(valLeft)) + static_cast<long
long>(getAsDouble(valRight));
        if (res > 2147483647LL || res < -2147483648LL)
throw NumericalOverflowException();
        memory.push(static_cast<int>(res)); // Hasil Int
    }
    break;
}
```

Unit logika dirancang untuk mendukung resolusi tipe data saat *runtime*. ALU tidak hanya dapat mengeksekusi operasi matematika pada bilangan bulat, tetapi juga mampu memproses bilangan riil, melakukan konversi ASCII implisit untuk

karakter, serta mengeksekusi *concatenation* dan komparasi leksikografis pada *string*.

3.2.4 Penanganan Kesalahan Saat *Runtime* (*Exception Handling*)

RuntimeStack.cpp

```
void RuntimeStack::push(std::variant<int, double, char,
std::string> value) {
    if (static_cast<int>(stack_.size()) >=
static_cast<int>(MAX_STACK_SIZE)) {
        throw StackOverflowException();
    }
    stack_.push_back(value);
}

std::variant<int, double, char, std::string>
RuntimeStack::pop() {
    if (stack_.empty()) {
        throw StackUnderflowException();
    }
    auto val = stack_.back();
    stack_.pop_back();
    return val;
}

//...

// Pembangkitan kode untuk menghitung alamat elemen array
saat runtime
emitLda(levelDiff, baseAddress); // Taruh alamat dasar di
stack

// ... loop komponen indeks ...
indexExpression->accept(this); // Evaluasi indeks (misal
'i') ke stack
if (low != 0) {
    emitLit(low);
    emitOpr(OprCode::SUB); // Sesuaikan dengan batas bawah
}
// ... kalikan dengan ukuran elemen ...
emitOpr(OprCode::ADD); // Tambahkan offset ke alamat dasar
```

Mesin virtual ini dilengkapi dengan lapisan pertahanan berlapis untuk mencegah kerusakan memori. Kelas *exception* khusus dibuat secara otomatis ketika sistem mendeteksi anomali, seperti *Stack Overflow*, *Stack Underflow*, *Out-of-Bounds*, hingga pembagian dengan nol dan *Numerical Overflow*.

Melalui instruksi CHK, mesin virtual akan memvalidasi nilai indeks yang dievaluasi secara dinamis terhadap deklarasi batas bawah dan atas *array*. Jika indeks berada di luar rentang, mesin akan langsung melemparkan *MemoryAccessException*.

Bab 4

Pengujian

Pengujian 1

1. Nama Pengujian : Array Assignment dengan Indeks Literal
2. Input file : valid10_array_static_index.txt
3. Output file :
 - a. Intermediate Code : valid10_array_static_index-intermediate-code.txt
 - b. Output : valid10_array_static_index-output.txt
4. Deskripsi : Pengujian ini mengecek akses array dengan indeks literal. Karena generator saat ini mendukung offset array statis, indeks seperti arr[1], arr[2], dan arr[3] harus diterjemahkan menjadi address memori berbeda.
5. Hasil Pengujian :

Tabel 4.1 Pengujian 1

Input	Output
program TestArrayStaticIndex; var arr: array[1..3] of integer; total: integer; begin arr[1] := 10; arr[2] := 20; arr[3] := 30; total := arr[1] + arr[2] + arr[3];	=== Intermediate Code === 0 INT 0 7 1 LDA 0 3 2 LIT 0 10 3 STI 0 0 4 LDA 0 3 5 LIT 0 1 6 OPR 0 2 7 LIT 0 20 8 STI 0 0 9 LDA 0 3 10 LIT 0 2 11 OPR 0 2 12 LIT 0 30

<pre> writeln('total = ', total); end. </pre>	<pre> 13 STI 0 0 14 LDA 0 3 15 LDI 0 0 16 LDA 0 3 17 LIT 0 1 18 OPR 0 2 19 LDI 0 0 20 OPR 0 2 21 LDA 0 3 22 LIT 0 2 23 OPR 0 2 24 LDI 0 0 25 OPR 0 2 26 STO 0 6 27 LIT 0 "total = " 28 OPR 0 13 29 LOD 0 6 30 OPR 0 14 31 RET 0 0 </pre> Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid10_array_static_index-intermediate-code.txt === Interpreter === total = 60 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid10_array_static_index-output.txt
---	--

Pengujian 2

1. Nama Pengujian : Array Access dengan Dynamic Index
2. Input file : valid11_array_dynamic_index.txt
3. Output file :
 - a. Intermediate Code : valid11_array_dynamic_index-intermediate-code.txt
 - b. Output : valid11_array_dynamic_index-output.txt
4. Deskripsi : Pengujian ini mengecek akses array menggunakan indeks dinamis yang berasal dari iterator loop i
5. Hasil Pengujian :

Tabel 4.2 Pengujian 2

Input	Output
program TestArrayDynamicIndex; var arr: array[1..3] of integer; i, total: integer; begin i := 2; arr[1] := 10; arr[i] := 20; arr[3] := 30; total := arr[1] + arr[i] + arr[3]; writeln('total = ', total); end.	=== Intermediate Code === 0 INT 0 8 1 LIT 0 2 2 STO 0 6 3 LDA 0 3 4 LIT 0 10 5 STI 0 0 6 LDA 0 3 7 LOD 0 6 8 CHK 1 3 9 LIT 0 1 10 OPR 0 3 11 OPR 0 2 12 LIT 0 20 13 STI 0 0 14 LDA 0 3 15 LIT 0 2 16 OPR 0 2 17 LIT 0 30

	<pre>18 STI 0 0 19 LDA 0 3 20 LDI 0 0 21 LDA 0 3 22 LOD 0 6 23 CHK 1 3 24 LIT 0 1 25 OPR 0 3 26 OPR 0 2 27 LDI 0 0 28 OPR 0 2 29 LDA 0 3 30 LIT 0 2 31 OPR 0 2 32 LDI 0 0 33 OPR 0 2 34 STO 0 7 35 LIT 0 "total = " 36 OPR 0 13 37 LOD 0 7 38 OPR 0 14 39 RET 0 0</pre>
--	---

Intermediate code berhasil disimpan di:

../test/milestone-4/output/valid11_array_dynamic_index-intermediate-code.txt

=== Interpreter ===

total = 60

Hasil interpreter berhasil disimpan di:

../test/milestone-4/output/valid11_array_dynamic_index-

	output.txt
--	------------

Pengujian 3

1. Nama Pengujian : Record Field Assignment dan Access
2. Input file : valid9_record_access.txt
3. Output file :
 - a. Intermediate Code : valid9_record_access-intermediate-code.txt
 - b. Output : valid9_record_access-output.txt
4. Deskripsi : Pengujian ini mengecek apakah generator dapat menghitung offset field record secara statis. Field p.x dan p.y harus memiliki offset berbeda di memori, lalu dapat digunakan dalam ekspresi.
5. Hasil Pengujian :

Tabel 4.3 Pengujian 3

Input	Output
program TestRecord; type Point == record x: integer; y: integer end; var p: Point; total: integer; begin p.x := 3; p.y := 4; total := p.x + p.y; writeln('total = ', total);	=== Intermediate Code === 0 INT 0 6 1 LDA 0 3 2 LIT 0 3 3 STI 0 0 4 LDA 0 3 5 LIT 0 1 6 OPR 0 2 7 LIT 0 4 8 STI 0 0 9 LDA 0 3 10 LDI 0 0 11 LDA 0 3 12 LIT 0 1 13 OPR 0 2 14 LDI 0 0 15 OPR 0 2 16 STO 0 5

end.	<pre>17 LIT 0 "total = " 18 OPR 0 13 19 LOD 0 5 20 OPR 0 14 21 RET 0 0 Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid9_record_access-interme diate-code.txt === Interpreter === total = 7 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid9_record_access-output.t xt</pre>
------	--

Pengujian 4

1. Nama Pengujian : Recursive Function Factorial
2. Input file : valid8_recursive_factorial.txt
3. Output file :
 - a. Intermediate Code : valid8_recursive_factorial-intermediate-code.txt
 - b. Output : valid8_recursive_factorial-output.txt
4. Deskripsi : Pengujian ini mengecek pemanggilan function secara rekursif. Test ini memvalidasi bahwa interpreter dapat membuat stack frame baru untuk setiap pemanggilan function, menyimpan return address dengan benar, lalu kembali ke caller setelah RET.
5. Hasil Pengujian :

Tabel 4.4 Pengujian 4

Input	Output
program	=== Intermediate Code ===
TestRecursiveFactorial;	0 INT 0 4
	1 JMP 0 19
var	2 INT 0 4
result: integer;	3 LOD 0 -1
	4 LIT 0 0
function fact(n: integer):	5 OPR 0 7
integer;	6 JPC 0 10
begin	7 LIT 0 1
if n == 0 then	8 STO 0 3
fact := 1	9 JMP 0 17
else	10 LOD 0 -1
fact := n * fact(n - 1);	11 LOD 0 -1
end;	12 LIT 0 1
	13 OPR 0 3
begin	14 CAL 1 2
result := fact(5);	15 OPR 0 4

<pre>writeln('fact = ', result); end.</pre>	<pre>16 STO 0 3 17 LOD 0 3 18 RET 1 1 19 LIT 0 5 20 CAL 0 2 21 STO 0 3 22 LIT 0 "fact = " 23 OPR 0 13 24 LOD 0 3 25 OPR 0 14 26 RET 0 0 Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid8_recursive_factorial-intermedia te-code.txt === Interpreter === fact = 0 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid8_recursive_factorial-output.txt</pre>
---	---

Pengujian 5

1. Nama Pengujian : Multiple Function Call dalam Satu Ekspresi
2. Input file : valid7_function_expression.txt
3. Output file :
 - a. Intermediate Code : valid7_function_expression-intermediate-code.txt
 - b. Output : valid7_function_expression-output.txt
4. Deskripsi : Pengujian ini mengecek apakah function call dapat digunakan sebagai ekspresi dan apakah stack dibersihkan dengan benar setelah setiap pemanggilan function. Tanpa cleanup parameter yang benar, hasil ekspresi ini bisa salah karena nilai argumen lama masih tertinggal di stack.
5. Hasil Pengujian :

Tabel 4.5 Pengujian 5

Input	Output
program TestFunctionExpression; var x: integer; function Add(a: integer; b: integer): integer; begin Add := a + b; end; begin x := Add(1, 2) + Add(4, 5); writeln('x = ', x); end.	=== Intermediate Code === 0 INT 0 4 1 JMP 0 9 2 INT 0 4 3 LOD 0 -2 4 LOD 0 -1 5 OPR 0 2 6 STO 0 3 7 LOD 0 3 8 RET 1 2 9 LIT 0 1 10 LIT 0 2 11 CAL 0 2 12 LIT 0 4 13 LIT 0 5 14 CAL 0 2 15 OPR 0 2

16 STO 0 3

17 LIT 0 "x = "

18 OPR 0 13

19 LOD 0 3

20 OPR 0 14

21 RET 0 0

Intermediate code berhasil disimpan di:

../test/milestone-4/output/valid7_function_expression-intermediate-code.txt

=== Interpreter ===

x = 5

Hasil interpreter berhasil disimpan di:

../test/milestone-4/output/valid7_function_expression-output.txt

Pengujian 6

1. Nama Pengujian : Procedure Call dengan Parameter
2. Input file : valid6_procedure_call.txt
3. Output file :
 - a. Intermediate Code : valid6_procedure_call-intermediate-code.txt
 - b. Output : valid6_procedure_call-output.txt
4. Deskripsi : Pengujian ini mengecek pemanggilan procedure dengan parameter. Test ini penting setelah perbaikan RET, karena procedure tidak boleh mengembalikan nilai, tetapi tetap harus membersihkan parameter dari stack setelah selesai.
5. Hasil Pengujian :

Tabel 4.6 Pengujian 6

Input	Output
program TestProcedure; var x: integer; procedure SetX(a: integer); begin x := a; end; begin SetX(7); writeln('x = ', x); end.	=== Intermediate Code === 0 INT 0 4 1 JMP 0 6 2 INT 0 3 3 LOD 0 -1 4 STO 1 3 5 RET 0 1 6 LIT 0 7 7 CAL 0 2 8 LIT 0 "x = " 9 OPR 0 13 10 LOD 0 3 11 OPR 0 14 12 RET 0 0 Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid6_procedure_call-intermediate-code.txt

	<pre>=== Interpreter === x = 7 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid6_procedure_call-output.txt</pre>
--	--

Pengujian 7

1. Nama Pengujian : Nested While Loop
2. Input file : valid4_nested_loop.txt
3. Output file :
 - a. Intermediate Code : valid4_nested_loop-intermediate-code.txt
 - b. Output : valid4_nested_loop-output.txt
4. Deskripsi : Pengujian ini mengecek kemampuan intermediate code generator menangani loop bersarang. Program memiliki while luar sebanyak 3 iterasi dan while dalam sebanyak 2 iterasi, sehingga total eksekusi blok terdalam adalah $3 * 2 = 6$.
5. Hasil Pengujian :

Tabel 4.7 Pengujian 7

Input	Output
program TestNestedLoop; var i, j, count: integer; begin i := 1; count := 0; while i <= 3 do begin j := 1; while j <= 2 do begin count := count + 1; j := j + 1; end; end; end; end;	=== Intermediate Code === 0 INT 0 6 1 LIT 0 1 2 STO 0 3 3 LIT 0 0 4 STO 0 5 5 LOD 0 3 6 LIT 0 3 7 OPR 0 12 8 JPC 0 29 9 LIT 0 1 10 STO 0 4 11 LOD 0 4 12 LIT 0 2 13 OPR 0 12 14 JPC 0 24 15 LOD 0 5 16 LIT 0 1

<pre> i := i + 1; end; writeln('count = ', count); end. </pre>	<pre> 17 OPR 0 2 18 STO 0 5 19 LOD 0 4 20 LIT 0 1 21 OPR 0 2 22 STO 0 4 23 JMP 0 11 24 LOD 0 3 25 LIT 0 1 26 OPR 0 2 27 STO 0 3 28 JMP 0 5 29 LIT 0 "count = " 30 OPR 0 13 31 LOD 0 5 32 OPR 0 14 33 RET 0 0 </pre> Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid4_nested_loop-intermediate-code.txt === Interpreter === count = 6 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid4_nested_loop-output.txt
---	--

Pengujian 8

1. Nama Pengujian : While Loop dengan Counter
2. Input file : valid3_while_loop.txt
3. Output file :
 - a. Intermediate Code : valid3_while_loop-intermediate-code.txt
 - b. Output : valid3_while_loop-output.txt
4. Deskripsi : Pengujian ini mengecek apakah generator menghasilkan alur perulangan while dengan benar. Kondisi harus dicek di awal loop, lalu menggunakan JPC untuk keluar dari loop dan JMP untuk kembali ke pengecekan kondisi.
5. Hasil Pengujian :

Tabel 4.8 Pengujian 8

Input	Output
program TestWhileLoop;	=== Intermediate Code ===
	0 INT 0 5
var	1 LIT 0 1
i, total: integer;	2 STO 0 3
	3 LIT 0 0
begin	4 STO 0 4
i := 1;	5 LOD 0 3
total := 0;	6 LIT 0 5
	7 OPR 0 12
while i <= 5 do	8 JPC 0 18
begin	9 LOD 0 4
total := total + i;	10 LOD 0 3
i := i + 1;	11 OPR 0 2
end;	12 STO 0 4
	13 LOD 0 3
writeln('total = ', total);	14 LIT 0 1
end.	15 OPR 0 2
	16 STO 0 3

	<pre>17 JMP 0 5 18 LIT 0 "total = " 19 OPR 0 13 20 LOD 0 4 21 OPR 0 14 22 RET 0 0 Intermediate code berhasil disimpan di: ../test/milestone-4/output/valid3_while_loop-intermediate-code. txt === Interpreter === total = 15 Hasil interpreter berhasil disimpan di: ../test/milestone-4/output/valid3_while_loop-output.txt</pre>
--	--

Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

- Fase *Intermediate Code Generation* berhasil menerjemahkan representasi *Abstract Syntax Tree* menjadi sekumpulan instruksi P-Code yang valid, linear, dan teroptimasi. Mekanisme kalkulasi alamat untuk variabel global, lokal, dan parameter telah berjalan dengan presisi.
- Arsitektur mesin virtual (Interpreter) berhasil mengeksekusi instruksi P-Code secara akurat. Penggunaan *Runtime Stack* dinamis memungkinkan kompilator menangani alokasi memori kompleks, manajemen *Stack Frame* untuk prosedur bersarang, serta operasi lintas tipe data (angka, teks, dan karakter) secara *type-safe*.
- Implementasi *error handling* pada tahap *runtime* terbukti tangguh dalam memproteksi mesin virtual dari ancaman akses memori ilegal maupun rekursi tak terbatas, memastikan eksekusi program dihentikan dengan aman.
- Mesin virtual terbukti mampu menangani operasi memori tingkat lanjut, yaitu pengalamatan tidak langsung (*indirect addressing*). Dengan ditambahkannya instruksi LDA, LDI, dan STI, kompilator sukses mengeksekusi operasi array dengan indeks dinamis (*dynamic indexing*) sekaligus menjamin keamanan akses memori melalui *runtime bounds checking* (CHK).

5.2. Saran

Beberapa hal yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas program pada pengembangan selanjutnya adalah:

- Kompilator saat ini telah memiliki infrastruktur pengalamatan tidak langsung (melalui LDA, LDI, STI) yang digunakan untuk array dinamis. Pada iterasi selanjutnya, infrastruktur ini sangat siap untuk diperluas guna mendukung tipe data *pointer* secara eksplisit (seperti operator $\wedge$ di Pascal), yang dipadukan dengan alokasi

memori dinamis (*heap memory*) seperti *new* dan *dispose* untuk struktur data non-linear.

- Untuk menjadikan Arion sebagai bahasa pemrograman yang lebih matang, disarankan adanya penambahan *Standard Library* bawaan (seperti fungsi manipulasi *string* tingkat lanjut atau I/O berbasis *file*), serta perbaikan *code optimization* pada tahap *Intermediate Code* agar eksekusi P-Code berjalan lebih efisien dan cepat.

Lampiran

Tautan Repositori Github:

<https://github.com/Irvin-Tandiarrang-Sumual/STI-Tubes-IF2224-2026.git>

NIM	Nama	Deskripsi Tugas	Persentase Kontribusi
13524008	Agatha Tatianingseto	Kerja	20%
13524030	Irvin Tandiarrang Sumual	Kerja	20%
13524074	Bernhard Aprillio Pramana	Kerja	20%
13524096	Moreno Syawali Ganda Sugita	Kerja	20%
13524110	Jennifer Khang	Kerja	20%

Daftar Referensi

GeeksforGeeks. (2026). *Phases of a Compiler*. [Online]. Available:

<https://www.geeksforgeeks.org/phases-of-a-compiler/>. Accessed 29 May 2026.

GeeksforGeeks. (2026). *Intermediate Code Generation in Compiler Design*. [Online]. Available:

<https://www.geeksforgeeks.org/intermediate-code-generation-in-compiler-design/>.

Accessed 29 May 2026.

GeeksforGeeks. (2025). *Stack based CPU Organization*. [Online]. Available:

[https://www.geeksforgeeks.org/computer-organization-architecture/introduction-of-stack-](https://www.geeksforgeeks.org/computer-organization-architecture/introduction-of-stack-based-cpu-organization/)

[based-cpu-organization/](https://www.geeksforgeeks.org/computer-organization-architecture/introduction-of-stack-based-cpu-organization/). Accessed 29 May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Intermediate Code Generation*. [Online]. Available:

https://www.tutorialspoint.com/compiler_design/compiler_design_intermediate_code_gen

[erations.htm](https://www.tutorialspoint.com/compiler_design/compiler_design_intermediate_code_gen). Accessed 29 May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Symbol Table & Memory Management*. [Online].

Available:

https://www.tutorialspoint.com/compiler_design/compiler_design_symbol_table.htm.

Accessed 29 May 2026.

cppreference.com. (n.d.). *std::variant - C++ Reference*. [Online]. Available:

<https://en.cppreference.com/w/cpp/utility/variant>. Accessed 4 June 2026.

GeeksforGeeks. (2025). *Type Checking in Compiler Design*. [Online]. Available:

<https://www.geeksforgeeks.org/type-checking-in-compiler-design/>. Accessed 4 June

2026.

W3Schools. (n.d.). *C++ Strings and Data Types*. [Online]. Available:

https://www.w3schools.com/cpp/cpp_strings.asp. Accessed 4 June 2026.

GeeksforGeeks. (2026). Three Address Code in Compiler Design. [Online]. Available:
<https://www.geeksforgeeks.org/three-address-code-compiler/>. Accessed 4 June 2026.